AI Compiler Performance Engineering

Auto-Loops Books — ch27

Contents

1 Future Trends in AI Compilers **1**

1.1 codesign 2

1.2 new architectures 4

1.3 edge cloud 5

1.4 autonomous kernels 6

1.5 Key Takeaways 8

1.6 Conclusion 9

Bibliography **10**

Chapter 1

Future Trends in AI Compilers

Trend chapters fail when they read like keynote slides—buzzwords without counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Part VII established production packaging and gates; this closing chapter projects **where AI compilers go next** while keeping Chapter 1's decode metrics as the constitution: bytes/token, launches/token, and residency legality across GPU, CPU, and NPU fleets (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet tomorrow's silicon adds new memory tiers and routing paths that obsolete today's fusion maps (Chen, 2026; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a). Trends matter only when they name *which counter moves* and *which gate breaks* (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Four trajectories close Part VII: **hardware–software co-design**, **new architecture adaptation**, **edge–cloud unified compilation**, and **autonomous kernel generation with legality gates** (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Goodfellow *et al.*, 2016). Part VIII (Chapters 28–30) implements the three-party contract among inference frameworks, YiRage runtime, and compiler co-design that these trends converge toward (Chen and YiRage

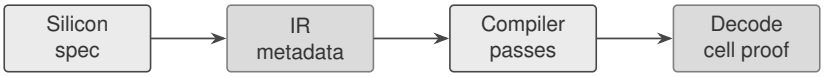


Figure 1.1: Co-design trend: silicon capabilities encoded as IR legality → compiler passes → decode-cell proof (not FLOPs alone).

Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Trend chapters fail operationally when teams adopt silicon before IR contracts exist—the same failure mode as shipping eager fallbacks after graph capture (NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Each section below names a trend, the counter it should move, and the gate that must not break (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Anti-patterns for trend adoption: chasing peak TFLOPs without decode cells; porting GEMM first; treating autonomous kernels as exempt from legality; merging edge and cloud fingerprints (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The fix is always the same methodology from Chapter 22—profile, gate, ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

1.1 codesign

Hardware–software co-design stops being a slogan when ISA and memory hierarchy changes ship with *compiler metadata contracts*—what can be fused, what must stay paged, which tiers exist (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hopper-class TMA and warp specialization

Trend	Counter moved	Gate
Co-design	bytes/token residency	IR legality oracle
New arch	launches/token on port	decode-cell regression
Edge–cloud	ms/token incl. RTT	tier fingerprint isolation
Autonomous kernels	search throughput	legality before profile

Table 1.1: Future trends mapped to decode counters and non-negotiable gates.

pushed fusion boundaries; the next co-design wave encodes those boundaries in IR so backends do not rediscover them per vendor (NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Co-design for *decode* differs from training: batch-1 residency, paging, and graph capture dominate; co-design documents must lead with bytes/token impact, not peak TFLOPs (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2023; NVIDIA, 2024, 2022). YiRage-style iron rules (Chapter 2) foreshadow vendor-neutral co-design: checkable predicates shipped alongside silicon (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Vendor roadmaps will keep adding memory tiers, expert routing, and capture modes—co-design is the process of turning those into compiler-visible facts before backends guess (SemiAnalysis, 2024; Liu *et al.*, 2024; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Multi-hardware delta. GPU co-design chases bandwidth and launch orchestration; NPU co-design chases static DMA depth; CPU co-design chases cache residency—shared methodology, incomparable constraints (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 1.1.1. New tier exposes faster on-chip buffer: co-design wins only

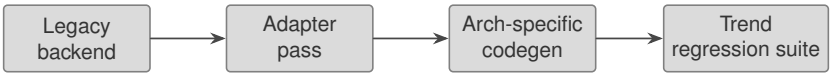


Figure 1.2: New architecture trend: adapter passes bridge legacy IR to new codegen with arch-specific decode regression suites.

if compiler IR marks which tensors may legally reside there at BS=1 decode (SemiAnalysis, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

1.2 new architectures

New architecture adaptation is the recurring tax every AI compiler team pays—MLIR dialects, backend forks, and HardwareRegistry entries—when silicon generations turn over (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). The anti-pattern is "port GEMM first, decode later"; decode-first ports expose paging and graph legality early (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Trend: modular adapter passes per architecture generation, each gated by the same decode cells from Chapter 22 (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Architecture trend lines without regression suites become marketing, not engineering (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

MoE-heavy architectures (DeepSeek-class) accelerate the trend: expert routing and disaggregated pools become first-class IR concepts, not runtime hacks (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors,

2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler teams should budget one adapter generation per silicon refresh—trying to stretch a Hopper-only fusion map across Blackwell and XDNA without adapter passes guarantees launch explosions (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Review gate. New architecture support merges only when bytes/token and launches/token match or beat prior generation on agreed buckets (Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.3 edge cloud

Edge–cloud unified compilation targets one model artifact with tier-specific fingerprints—edge NPU static schedules, cloud GPU graph+capture, CPU fallback—without pretending one schedule fits all (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The edge trend is longer context on device; the cloud trend is fleet goodput—unified IR must express both without lowest-common-denominator fusion (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Adaptive compilation at runtime (detect chip, select backend) is the operational face of this trend—HardwareRegistry patterns from YiRage generalize to edge fleets (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Cloud offload decisions (which layers run edge vs cloud) belong in policy metadata compiled into the same IR family (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Privacy and bandwidth drive edge trends, but **decode counters** still arbitrate: edge wins only if ms/token and bytes/token beat cloud RTT-inclusive baselines for the target workload (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Unified edge–cloud compile pipelines version fingerprints per tier and forbid silent cross-tier promotion (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Fleet operators increasingly run hybrid graphs: prefill on cloud GPU, decode on edge NPU, or the reverse for privacy-sensitive prompts (AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Unified compilation must emit *split fingerprints*—not one schedule pretending both tiers share memory (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Example 1.3.1. Same LLM runs on XDNA edge and Hopper cloud: two fingerprint namespaces, one IR export, shared decode-cell methodology (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

1.4 autonomous kernels

Autonomous kernel generation—LLM-assisted or search-driven—is the most hyped trend and the highest risk (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Chapter 25 placed RL and auto-optimization under decode rewards; the trend extends toward models proposing fusion patterns or kernel text (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon

et al., 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Non-negotiable guardrails for autonomous kernels:

1. **Legality oracle before profile** — same as human-authored schedules (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

2. **Decode-cell reward** — bytes/token, launches/token, ms/token; no GEMM-only scores (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

3. **Human prior + fingerprint rollback** — autonomy subordinate to production gates (Chapter 26) (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Autonomous *kernel* text without IR integration repeats CUDA string injection failures—trend winners compile through the same multi-backend stacks Part VI built (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The autonomous trend succeeds when it accelerates search under existing gates, not when it bypasses them (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Search budgets for autonomous kernels should cap candidate evaluations per merge request—unbounded search is how illegal schedules reach main (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Chapter 25's RL autotune patterns extend here: reward decode goodput, penalize legality violations, require human-readable diffs (Chen and

YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

1.5 Key Takeaways

- **Co-design ships as compiler metadata contracts.** Hardware–software co-design stops being a slogan when an ISA or memory-hierarchy change arrives with a contract the compiler can read—what may be fused, what must stay paged, which tiers exist (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Encoding silicon capabilities as IR legality and proving each on decode cells is what turns a new feature into a usable, checkable optimization rather than a press release (Chen, 2020; Davies *et al.*, 2025).

- **New architectures are a recurring, decode-first tax.** Every silicon generation forces adapter passes, backend forks, and hardware-registry entries, and the discipline that keeps the cost bounded is decode-first porting with architecture-specific regression (Chen, 2020; Chen and YiRage Contributors, 2026). Bringing a new target up on prefill alone hides exactly the residency and launch gaps that batch-1 decode will later expose (Chen, 2026; Vellaisamy *et al.*, 2026).

- **Edge–cloud means one artifact, tier-specific fingerprints.** Unified compilation targets a single model with per-tier fingerprints—edge NPU static schedules, cloud GPU graph capture, CPU fallback—without pretending one schedule fits all (AMD, 2024b,a). Adaptive backend selection at runtime with no cross-tier promotion is what prevents a schedule tuned for one tier from silently regressing another (Davies *et al.*, 2025; Kwon *et al.*, 2023).

- **Autonomous kernels need legality, reward, and rollback.** LLM-assisted and search-driven kernel generation is the most hyped and highest-risk trend, because a generated kernel can be fast and wrong (Chen, 2020; Chen and YiRage Contributors, 2026). It is safe only inside a frame that verifies legality, rewards decode goodput rather than microbench latency, and can roll back—treating the generator as a hint source, not a bypass of the residency contract (Chen, 2026; Davies *et al.*, 2025).

- **Part VIII makes these trends deployable.** The co-design, portability, and automation trends above are not abstractions; framework, runtime, and

compiler co-design is where they become a shipped serving stack (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023). The next part descends from trend to implementation, carrying the same decode-goodput yardstick (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.6 Conclusion

Future trends in AI compilers converge on one discipline: faster silicon and smarter automation must still answer to decode goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Co-design, new architectures, edge–cloud unity, and autonomous kernels are directions—not excuses to drop bytes/token gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Part VIII opens with inference frameworks (Chapter 28), YiRage PersistentKernel runtime (Chapter 29), and three-party co-design (Chapter 30) (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—the trend line starts where Chapter 1 started (SemiAnalysis, 2024; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 3, 4, 5, 6, 7, 8, 9

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 6, 7, 8, 9

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 2, 3, 4, 5, 6, 7, 8, 9

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 9

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 7, 8, 9

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 4, 5, 6, 7, 8, 9

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 2, 3, 4, 5, 6, 7, 8, 9

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 6, 7, 8, 9

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 6, 7, 8, 9

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9